Introduction

Peerapon S.

CPE 231 Algorithms

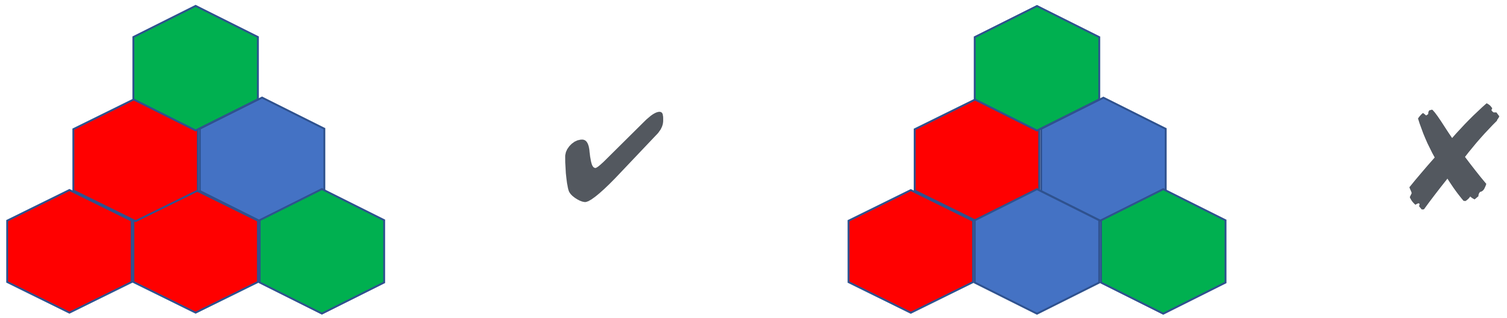
Topics

Algorithm definition and properties

Expressing algorithms

Quilt of hexagonal patches

Each hexagon and two beneath it must have the same color or three different colors.

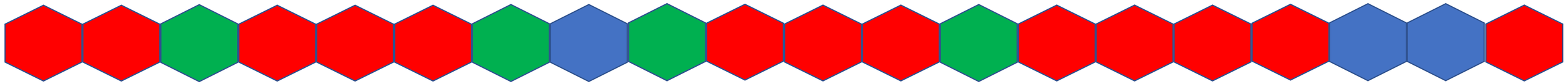


The Australian Informatics Competition, David CLARK, Mike CLAPPER
Olympiads in Informatics, 2014, Vol. 8, 179–189 179 © 2014 Vilnius University, IOI

Self Test: Quilt of Hexagonal Patches

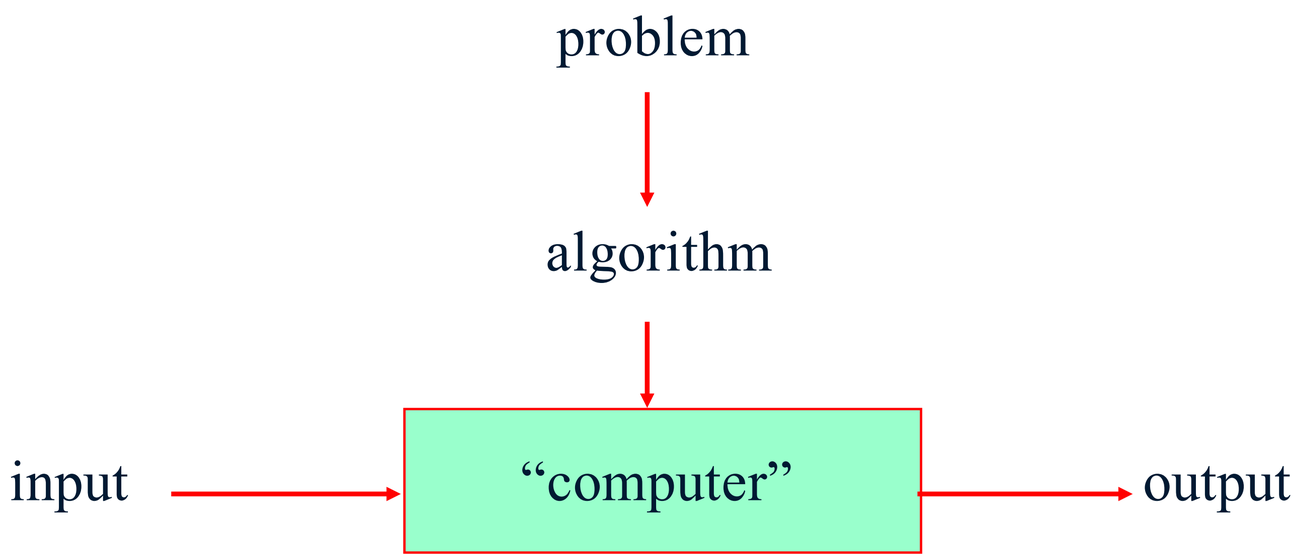
https://forms.cloud.microsoft/r/CCHSywMuP0?origin=lprLink

How many blue patches will there be?



What is an Algorithm?

A sequence of _____ *instructions* for solving a _____ *task or problem* in _____ *time*.



Procedure Get-Rich-Quick

Step 1: Get 40,000 THB

Step 2: Get a copy of tomorrow's newspaper and write down the closing values of today's stock.

Step 3: Go to a stock broker.

Step 4: Buy or sell those stocks appropriately.

Step 5: Take the profits and return to step 2.





D. Knuth, The Art of Computer Programming, 2nd edition,
Volume 1 - Fundamental Algorithms, 1973



Five Essential Properties of Algorithm

Finiteness

Definiteness

Input

Output

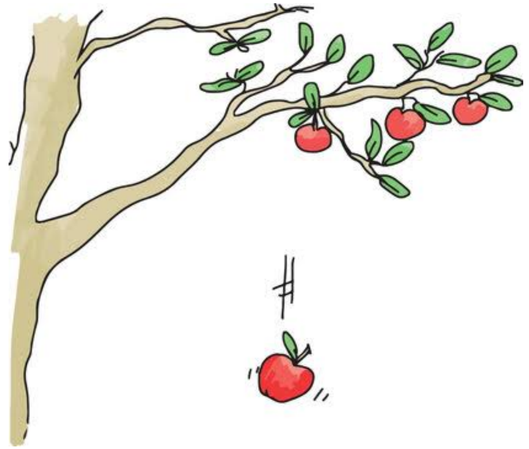
Effectiveness



Problem Abstraction and Modeling

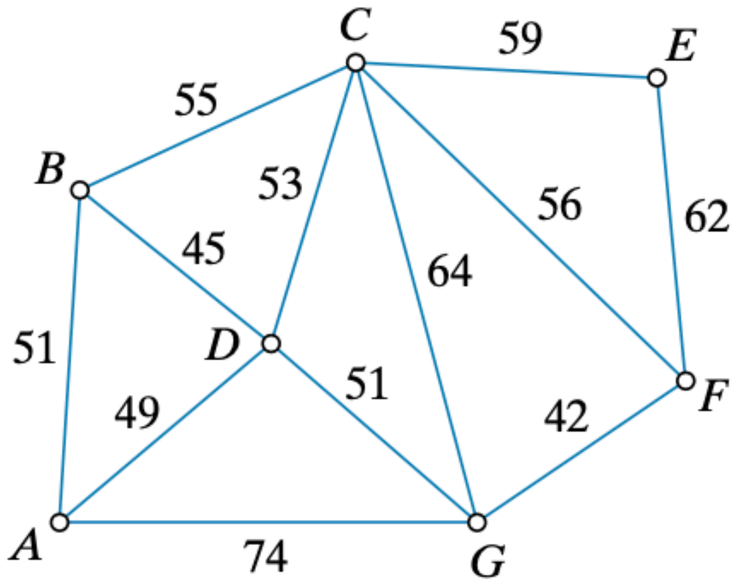
🍎 + 🍎 = 🍎🍎

🍌 + 🍌 = 🍌🍌



Remove distracting details and simplify
to focus on "core/generic patterns" of the problem


Groningen
Leeuwarden
Harlingen
Dokkum
Drachten
Sneek
Assen
Den Helder
Emmen
Meppel
Slagharen
Alkmaar
2 h 30 min
245 km
2 h 38 min
every 60 min
2 h 33 min
250 km
Deventer
Hengelo
Enschede
Leiden
Amersfoort
Apeldoorn
The Hague
Utrecht
Arnhem
Rotterdam
Nijmegen
Bocholt
Dordrecht
's Hertogenbosch
Weeze
Breda
Tilburg
Bergen
op Zoom
Eindhoven


A
B
C
D
E
F
G
55
59
53
56
62
45
64
51
51
49
42
74

Problem vs. Instance of Problem

Problem = A general abstract question or task defined by its inputs and expected outputs.

Example: Sorting problem

- **Input**: A sequence of n orderable keys $a_1, a_2, \ldots, a_n$
- **Output**: The permutation (reordering) of the input sequence such that

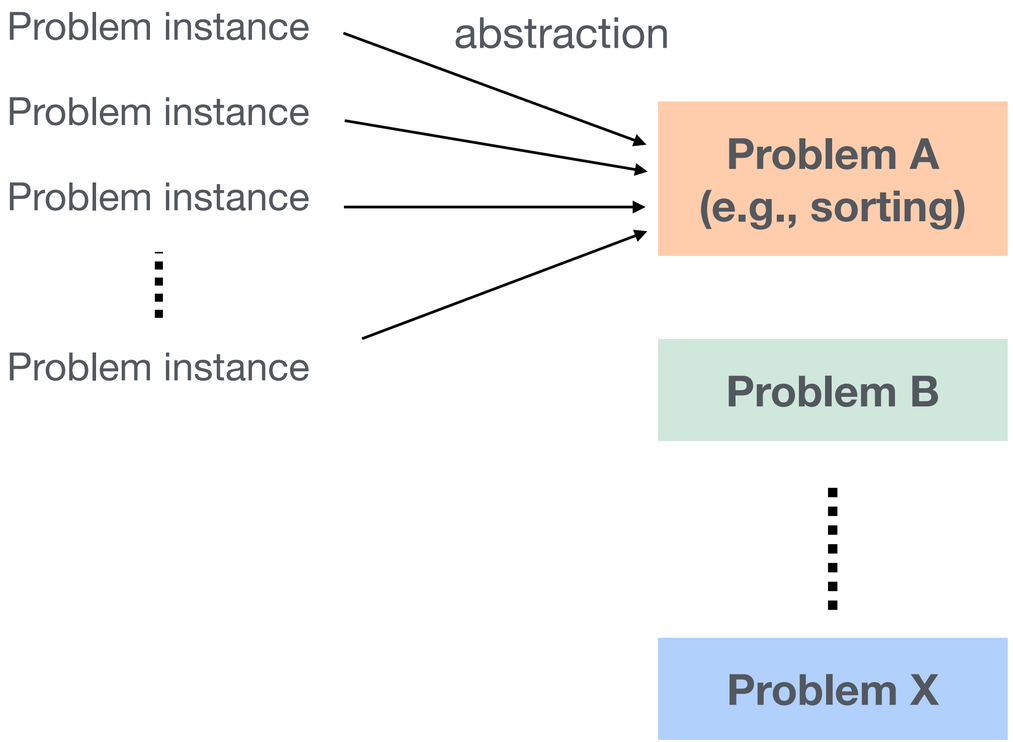
$$a'_1 \leq a'_2 \leq \cdots \leq a'_{n-1} \leq a'_n$$

Problem instance = "Real-world versions" of the problem

Example: Instance of sorting problem

- **Input**: Array of names {Mike, Bob, Sally, Jill, Jan}, or a list of numbers like {154, 245, 568, 324, 654, 324}
- **Output**: The sorted array or list above.

Real-world problems



Algorithm DTW(Q,C, w)

Input: $Q = (x_1, x_2, ..., x_m), C = (y_1, y_2, ..., y_n)$, and Constraint window w
Output: Minimum alignment cost

for $i = 1$ to n **do**
  for $j = 1$ to m **do**
    $F[i, j] \leftarrow \infty$
$F[0, 0] \leftarrow 0$

for $i = 1$ to n **do**
  for $j = \max(1, i-w)$ to $\min(m, i + w)$ **do**
    $F[i, j] \leftarrow 0$
for $i = 1$ to n **do**
  for $j = \max(1, i-w)$ to $\min(m, i + w)$ **do**
    $d_{ij} \leftarrow (x_i - y_j)^2$
    $F[i, j] \leftarrow d_{ij} + \min\{F[i-1, j], F[i, j-1], F[i-1, j-1]\}$

return $F[n, m]$

Order statistics	Weighted interval scheduling
Maximum subarray	Maximum matching
Closest pair of points	String matching
Shortest path	Longest-common subsequence
Spanning tree	Minimum alignment cost
Cycle detection	Subset sum
Strongly connected components	Traveling salesman
Articulation point	Graph coloring
Maximum flow	Hamiltonian path
Knapsack	etc.

Expressing Algorithm in Natural Languages

Algorithm MaxElement

1: Input: A sequence of numbers $a_1, a_2, \cdots, a_n$

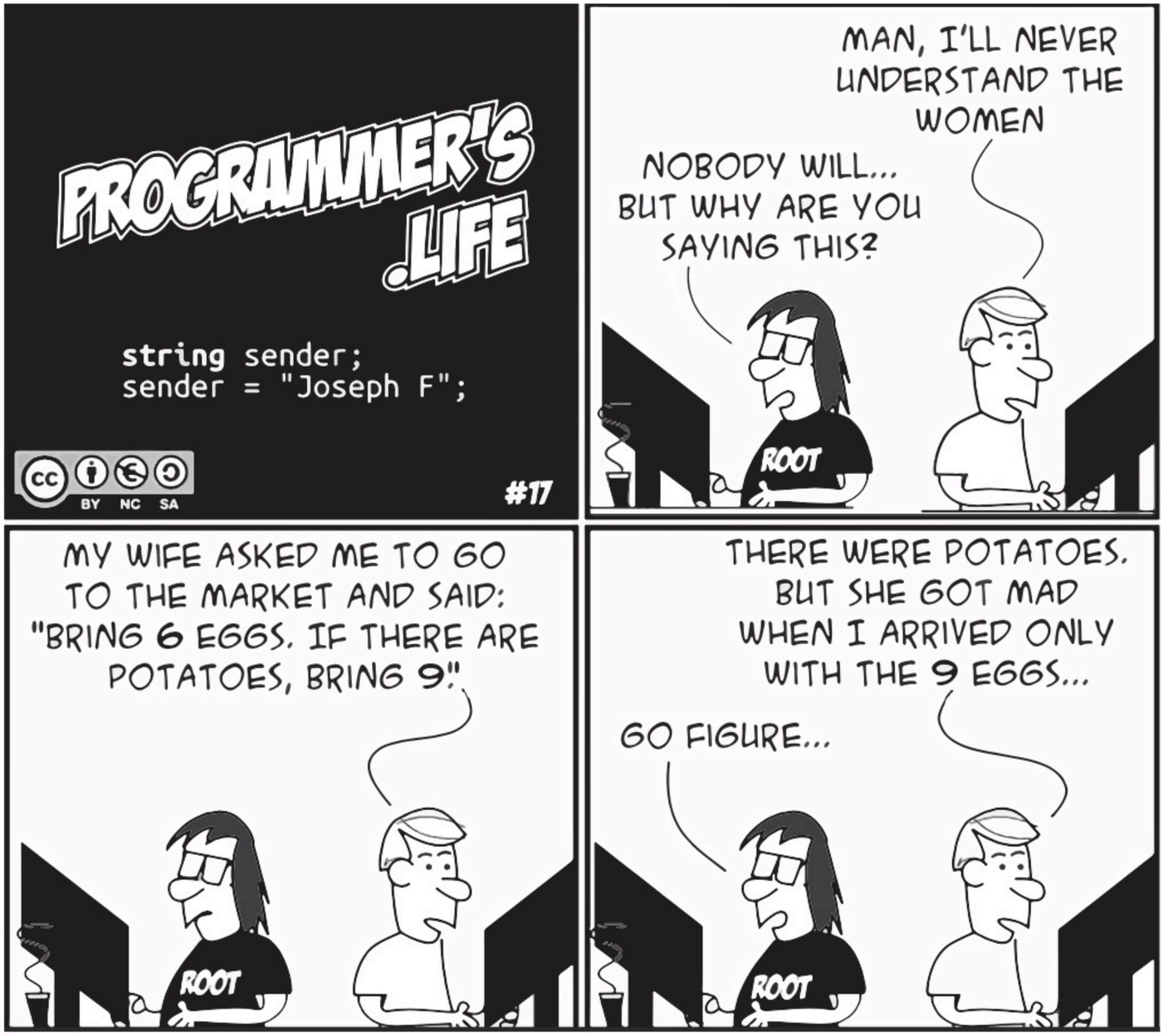
2: Output: The largest element in the input sequence

3:

4: **Step 1:** Set the first element as the current maximum

5: **Step 2:** For the rest of the elements, compare the current maximum to the each element. If the element is greater than the current maximum, update the current maximum to the current element.

6: **Step 3:** Take the current maximum as the algorithm output.



"Programmer's Logic", courtesy of http://programmers.life.

Expressing Algorithm in Programming Language

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
  int i, N, *a, maxval;

  printf("Enter the number of elements: ");
  scanf("%d", &N);
  a = malloc(N*sizeof(N));

  printf("Enter %d numbers: ", N);

  for(i=0;i<N;i++)
    scanf("%d", &a[i]);

  maxval = a[0];
  for(i=1;i<N;i++)
    if(maxval < a[i])
      maxval = a[i];

  printf("The largest element is %d\n", maxval);
}
```

```python
print "Enter a sequence of numbers:",
a = map(int, raw_input().split())

maxval = a[0]
for n in a:
    if maxval < n:
        maxval = n

print "The largest element is ", maxval
```

```javascript
function minMax(items) {
    return items.reduce(
        (accumulator, currentValue) => {
            return [
                Math.min(currentValue, accumulator[0]),
                Math.max(currentValue, accumulator[1])
            ];
        }, [Number.MAX_VALUE, Number.MIN_VALUE]
    );
}
```

Expressing Algorithm in pseudocode

Mix of natural language and programming-like language.

Algorithm MaxElement

1: Input: A sequence of numbers $a_1, a_2, \cdots, a_n$
2: Output: The largest element in the input sequence
3:
4: $maxval \leftarrow a_1$ ← Could use something like: **Assign a_1 to maxval**
5: **for** $i = 2$ to n **do**
6: **if** $maxval < a_i$ **then**
7: $maxval \leftarrow a_i$
8: Return $maxval$ ▷ $maxval$ is the largest element

Examples of Pseudocode Constructs

Algorithm MaxElement

1: Input: A sequence of numbers $a_1, a_2, \cdots, a_n$
2: Output: The largest element in the input sequence
3:

Header

4: $maxval \leftarrow a_1$ — Value assignment
5: **for** $i = 2$ to n **do** — Iteration
6: **if** $maxval < a_i$ **then** — Alternate path selection
7: $maxval \leftarrow a_i$
8: Return $maxval$ — Algorithm output

Comment: $\triangleright$ $maxval$ is the largest element

Properly indent your pseudocode.

Beware of loop index in iteration.

Header (Name, Input, Output)

Value assignment, comparisons, math operations ($+, -, \times, \div, \sqrt{}$, absolute, exponent, etc.)

Alternate path selection

- **if** (condition) ...

  else if (condition) ...

  else ....

Iterations (with **break**, **continue**)

- **for** (condition) ...
- **while** (condition) ...
- **do** ... while (condition)

```
Input: NIND, City, Citynum, Capacity, Demand, Distance, maxRoute, maxVehicle
Output: Chromosome of Population(NIND)

Initialize parameters
while i<=NIND
    TSProute=random location to individual route
    Random first UAV to use
    k = number of location +1
    for (1) loop j=2:number of location + 1
        k = k+1
        If used UAVs <= maxVehicle
            If number of travelled trip <= maxRoute
                If cumulative energy <= EnergyCapacity(index of usedUAV) && cumulative payload <= PayloadCapacity(index of usedUAV)
                    calculate energy and payload
                    store locations
                    If all locations were visited
                        store type of UAV to Chromosome
                    End if
                Else
                    If first visit location exceed the energy or payload capacity
                        Flag = 1
                        break
                    End if
                    trip = trip + 1
                    If number of trip >= maxRoute
                        If number of used UAV >= maxVehicle
                            Flag = 1
                            break
                        Else
                        Add UAV splitter (1,0,1) to chromosome
                        reset trip to 1
                        Add another UAV
                        Random new UAV that is not the same as previous UAV's type
                        calculate energy and payload
                        store locations
                        End if
                    End if
                    Add trip splitter
                    calculate energy and payload
                    store locations
                    store type of UAV to Chromosome
                End if
            End if
        Else
            Flag = 1
            break
        End if
    End for
    store route to Population (i)
    If (flag == 1)
        set Chromosome to null
        Flag = 0
        i = i+1
    Else
        i = i+1;
    End if
End while
```

Self Test: Expressing Algorithms



https://forms.cloud.microsoft/r/QuY8TPAsDF?origin=lprLink

In-Class Assignment L1-1: Quilt of Hexagonal Patches (Group of 2 to 3)

(25 minutes) Submit to LEB2

Summary

Algorithm not the same as procedure - Five essential properties

A mix of pseudocode and natural language to express an algorithm.

Real-world problems are reduced into their abstract form or problem.

Algorithms already exist to solve several a wide variety of problems.